

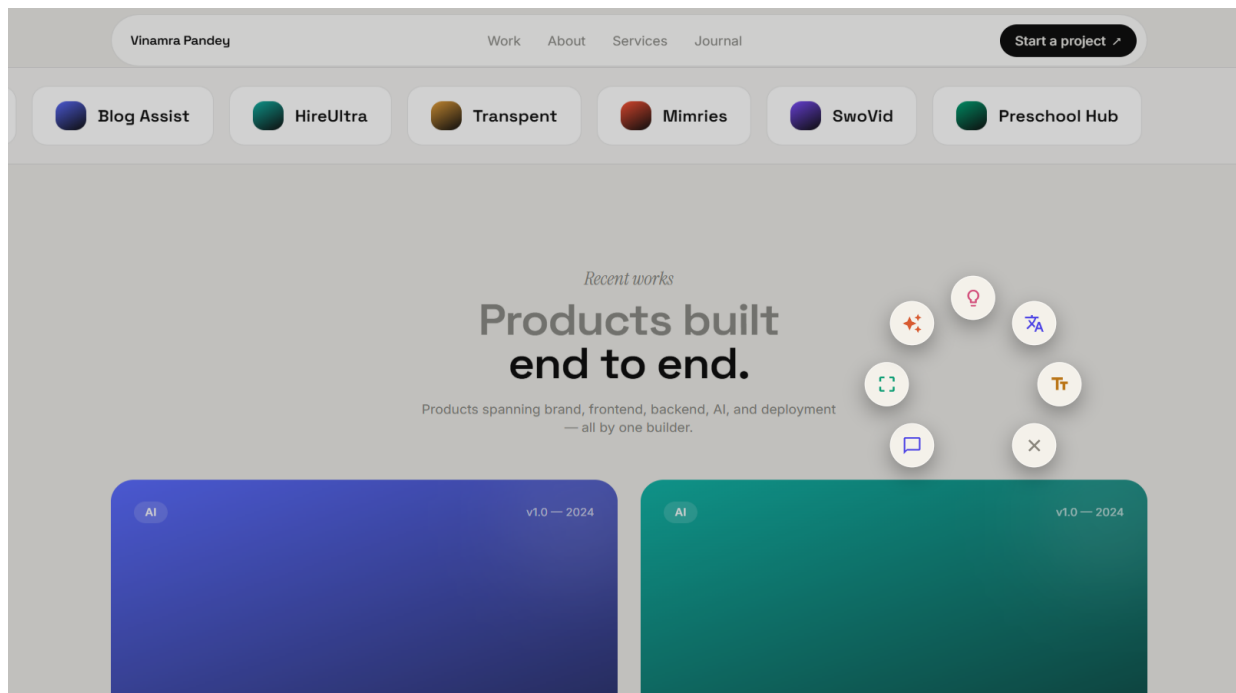


# Magisor: Case Study

*Building a Windows-native “Magic Pointer”: from a shake gesture to a shipped product, one specification-driven phase at a time with Claude Code.*

|            |                                                                                         |
|------------|-----------------------------------------------------------------------------------------|
| Author     | Vinamra Pandey                                                                          |
| Timeline   | ~7 weeks, May–July 2026                                                                 |
| Stack      | Flutter (Dart) + native C++/WinRT, on-device OCR, multi-provider AI                     |
| Status     | Shipping: v1.6.0 on Windows, macOS/iOS/Android planned                                  |
| Repository | <a href="https://github.com/vinamrapandey/Magisor">github.com/vinamrapandey/Magisor</a> |

*Shake it. It freezes. You act.*



*The pie menu open over a frozen desktop, mid-shake-invocation.*

## SECTION 01

### The problem

Every AI tool asks you to leave what you're doing to use it. Open a tab, paste your context, wait, come back. That friction isn't an accident. It's baked into the architecture of a chat window. Meanwhile, Android had shipped **Circle to Search**, Google was previewing an AI-native pointer concept internally called “**Magic Pointer**,” and

Windows had nothing like it.

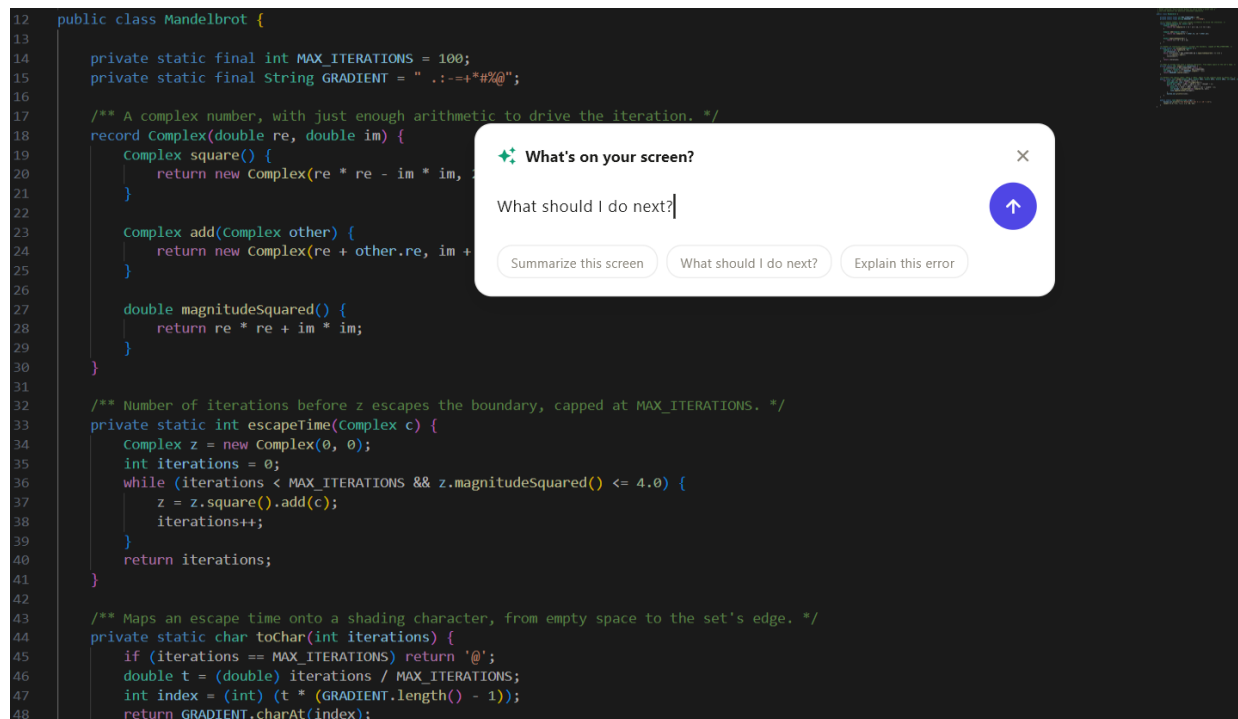
The brief I gave myself: *make AI feel like a native layer of the operating system, invoked by a physical gesture, that acts directly on the pixels I'm already looking at. No window switch, no copy-paste, no context loss.*

## SECTION 02

# The product

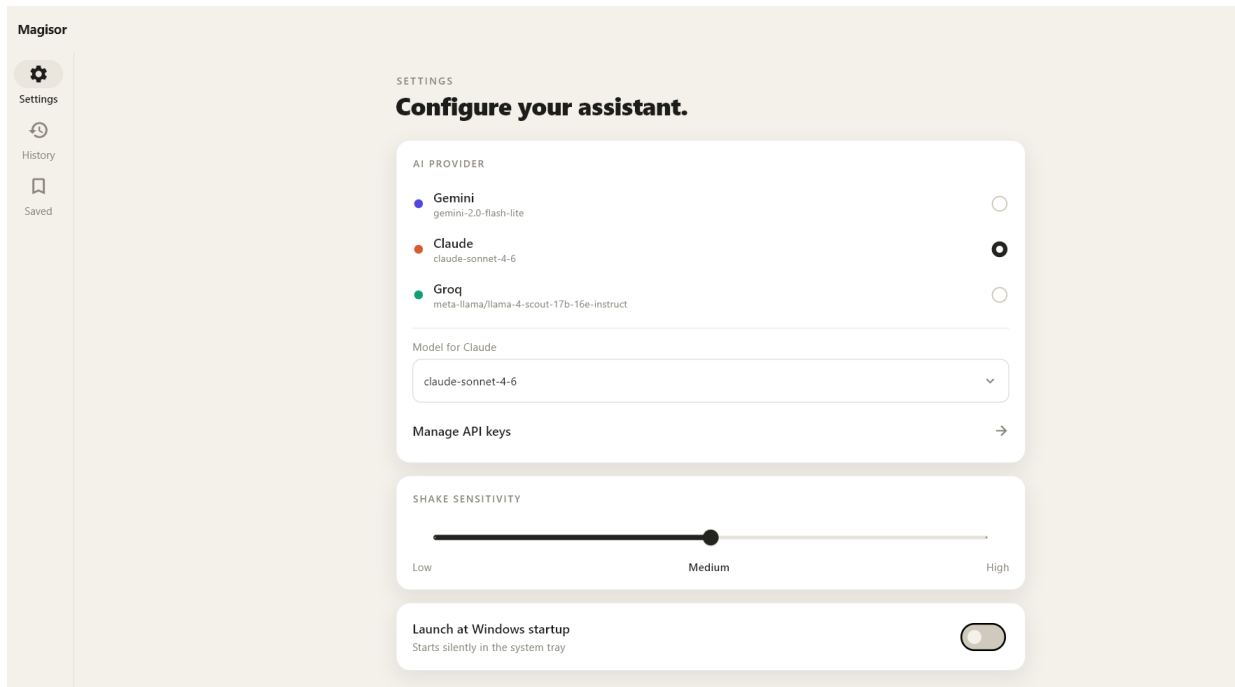
**Magisor** is an invisible AI companion that lives in your Windows system tray. **Shake your mouse** and the current screen freezes into an interactive overlay. From there:

- **Ask**: a free-form question about anything on screen, like “what does this error mean?” or “summarize this.”
- **Select** (Circle-to-Search): drag a box around any region and the AI analyzes just that.
- **Select Text**, universal on-device OCR: drag across any words, whether they're in an image, a video frame, a PDF, a game, or a locked-down UI, then Copy, Translate, Search, or Ask about that text.
- **Summarize / Explain / Translate**, one-tap actions on the whole frozen screen.
- Multi-turn follow-ups that keep asking about the same frozen context without re-explaining yourself.



The Ask bar in action, floating over a live code editor: “What's on your screen?” with quick-start suggestions (Summarize this screen, What should I do next?, Explain this error).

Every result is stored locally, and anything worth keeping can be starred. You bring your own key for Gemini, Claude, or Groq: no subscription, no hosted backend, no telemetry.



The dashboard's Settings screen: pick your AI provider (with a live accent-color dot per provider), choose the exact model, manage API keys, tune shake sensitivity, and toggle launch-at-startup. The editorial cream/charcoal design (v1.6.0) replaced an earlier dark-glass UI.

## Who it's for

- **Developers** who spend all day reading error messages, docs, and Stack Overflow. Explaining a stack trace is one shake away, with no context-switch to a browser tab.
- **Researchers and students** pulling text out of PDFs, papers, videos, and locked-down journal sites that don't let you select text normally.
- Anyone crossing a language barrier, with instant translation on any text visible anywhere on screen.
- Power users of Android's Circle to Search who've been asking for the desktop equivalent.

## How it's different (and what it isn't)

|            | Magisor                                                          | The usual alternative                          |
|------------|------------------------------------------------------------------|------------------------------------------------|
| Invocation | Physical gesture, in place, no window switch                     | Open an app, hit a shortcut, or alt-tab        |
| Works on   | Anything visible, including games, DRM video, images, locked UIs | Only apps exposing text via accessibility APIs |
| OCR        | On-device (Windows.Media.Ocr), no cloud round-trip               | Often a cloud vision call for every OCR step   |
| AI backend | Bring your own key: Gemini, Claude, or Groq                      | A single hosted model behind a subscription    |

|      | Magisor                                                     | The usual alternative                         |
|------|-------------------------------------------------------------|-----------------------------------------------|
| Data | Local SQLite, keys in the OS credential store, no telemetry | Often synced to a vendor's servers by default |

It is **not** a Copilot competitor: Copilot lives *inside* Microsoft apps, while Magisor is an overlay *across* all of them. It is **not** a ChatGPT wrapper, since there's no hosted service in between. It is **not** a screenshot tool: Snipping Tool copies a rectangle, Magisor understands what's inside it. And it is **not** a subscription. Bring your own key, local-first, free to run.

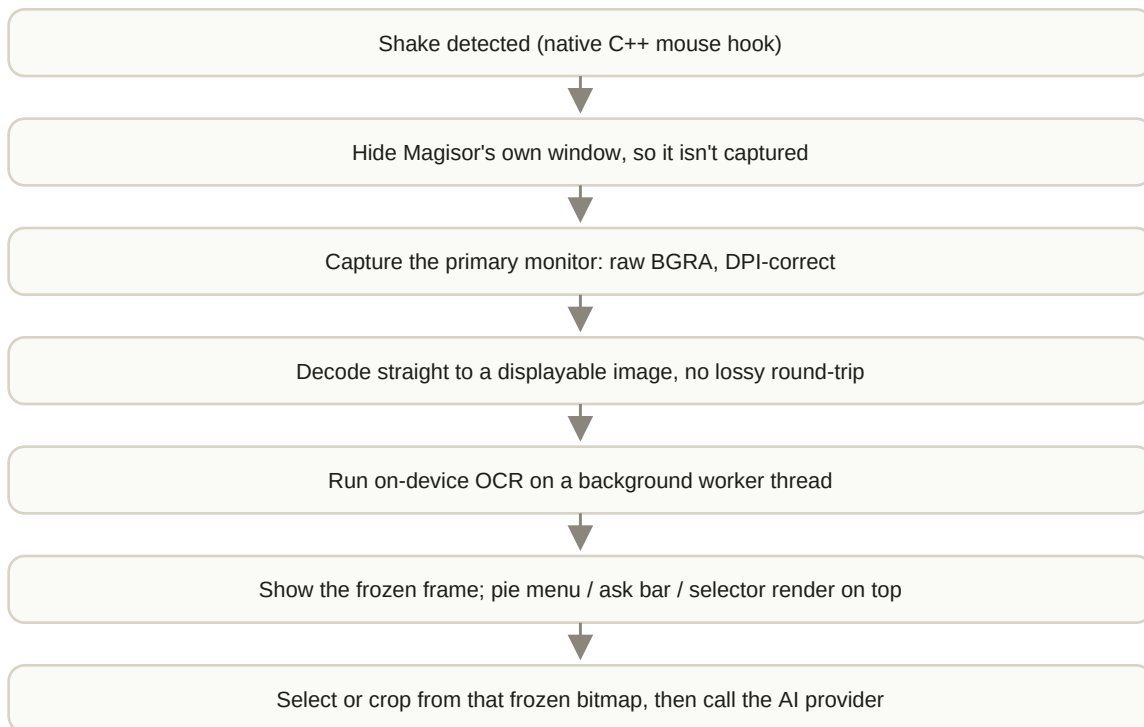
## SECTION 03

# Key technical decisions

Three decisions shaped everything else about how Magisor is built. None of them came from an AI suggestion; each came out of hands-on testing during the build (see Section 05 for how that division of labor actually worked).

## Decision 1: freeze the screen, not the live desktop

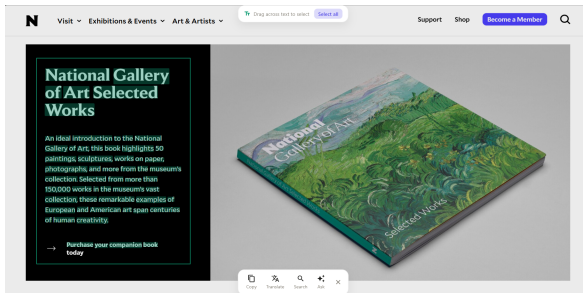
Windows has no equivalent of Android's overlay accessibility APIs, and no universal way to read text out of an arbitrary app. Games, DRM video, and many modern UI frameworks expose nothing to assistive APIs. Rather than fight that, Magisor's architecture leans into it: capture a still frame the instant the shake fires, then do every subsequent interaction against that static bitmap instead of the live, possibly-animating desktop.



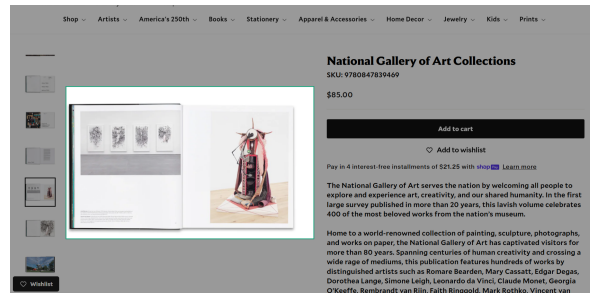
*The freeze-first pipeline: a shake triggers a capture, not a live read, and every later step operates on that one frozen frame.*

## The two don't compose.

That line is from the project's own spec doc, describing why a planned “read the live accessibility tree” mode (Section 04 covers it) was dropped: UI Automation reads the screen live, by coordinate, and Magisor's overlay sits on top of a frozen image. Once the decision to freeze was made, that mode was no longer compatible with it, on principle, not as a bug to fix.



*Text-select mode: dragging across words highlights them, with a Copy / Translate / Search / Ask toolbar at the bottom.*



*Circle-to-Search: a region drawn around a specific photo, ready to analyze just that selection.*

## Decision 2: on-device OCR over a cloud vision call

Text recognition runs locally through Windows.Media.Ocr rather than a cloud vision API. That keeps whatever's on screen off a third party's server by default, removes a network round-trip before the AI call even starts, and works offline. The trade-off is real: a cloud model can be more accurate on messy handwriting. For the overwhelming majority of real usage, printed and on-screen UI text, on-device OCR is fast and accurate enough, and the latency win matters more for a gesture that's supposed to feel instant.

## Decision 3: bring your own key, no hosted backend

Magisor stores whatever API key you paste, encrypted in the OS credential store, and calls Gemini, Claude, or Groq directly from the device. There's no sign-up flow, no usage-based billing to run, and no account system to build or secure. The cost is that Magisor has no free tier of its own; the benefit is that a one-person project can ship something that doesn't depend on anyone's hosted inference bill staying affordable.

### SECTION 04

## The build: from sketch to shipped product

### The origin sketch

Before a line of code, the whole concept lived in a single roadmap document: trigger is a physical gesture, surface is a radial menu at the cursor, freeze-first interaction, bring-your-own-key, local-first, Windows first because the platform gap was biggest. That document (`magisor_project_roadmap.md`) still lives in the repo, unedited from week one.

## Prototype to real product: the tech pivot

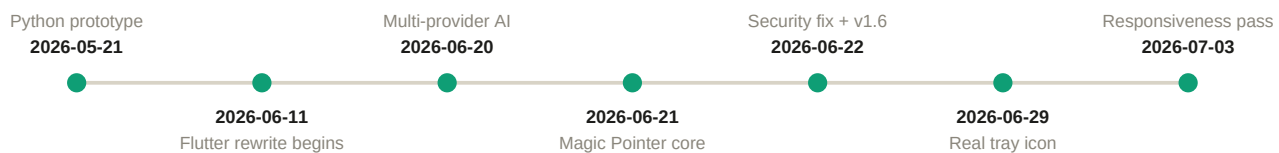
The first working version was **Python and PyQt5**. It proved the shake gesture and a Gemini vision pipeline worked, but wasn't going to deliver the UI quality this needed, and packaging Python for Windows distribution is genuinely painful. The call was to rewrite in Flutter and keep the native C++ mouse hook: Flutter gives a first-class rendering pipeline, and the method-channel boundary between Dart and C++ is exactly the seam this project wants, portable code in Dart, platform-specific code only where the OS demands it. Git history documents this as “Flutter revamp Steps 1 through 12.”

## The re-architecture moment

Partway through, live-testing surfaced something wrong: the overlay was transparent over the live desktop, so every interaction fought the fact that content underneath kept moving. Rather than patch around it, feature work paused for a full phased spec (`magic_pointer_spec.md`), covering the freeze-the-screenshot model from Section 03. Shipping that plan phase by phase, build, verify, commit, next phase, is the single biggest reason a ~7-week, one-person codebase doesn't read like a pile of half-finished retries.

One phase in that plan was explicitly dropped after prototyping: a UI-Automation-based mode meant to read exact text straight from the accessibility tree. As covered in Decision 1, that approach reads the live screen by coordinate, which doesn't compose with a topmost window sitting on a frozen image. The finding is written directly into the spec, so it's a documented decision, not a forgotten dead end.

## Release timeline

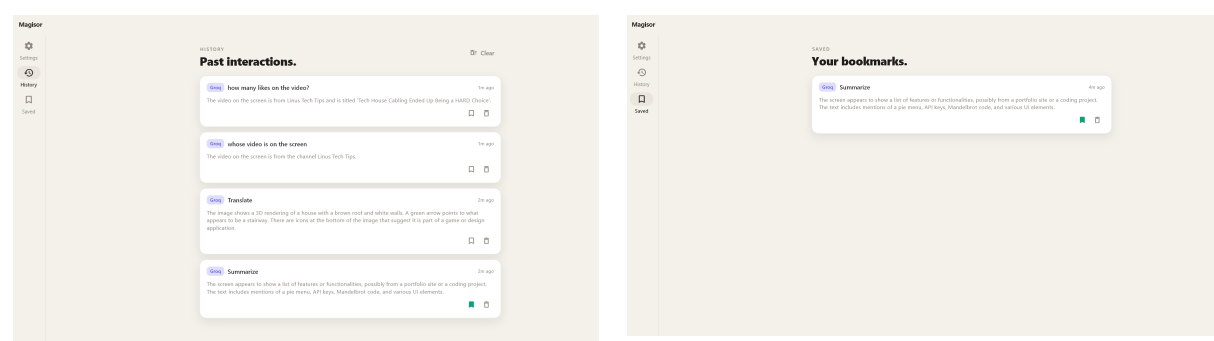


| Date       | Milestone                                                                                  |
|------------|--------------------------------------------------------------------------------------------|
| 2026-05-21 | Initial commit: Python/PyQt5 prototype, shake detection, tray app                          |
| 2026-06-11 | Flutter rewrite begins: Glass UI, auth, shake service, tray and pie-menu redesign          |
| 2026-06-20 | Multi-provider AI merged; Ask flow; Magic Pointer spec drafted                             |
| 2026-06-21 | Magic Pointer phases 1-3 done; UI Automation dropped (finding documented in the spec)      |
| 2026-06-22 | Security fix: leaked API key removed from repo; v1.6.0 editorial redesign (cream/charcoal) |
| 2026-06-29 | Real tray icon and working right-click menu                                                |
| 2026-07-03 | Desktop responsiveness pass (current)                                                      |

# Real numbers

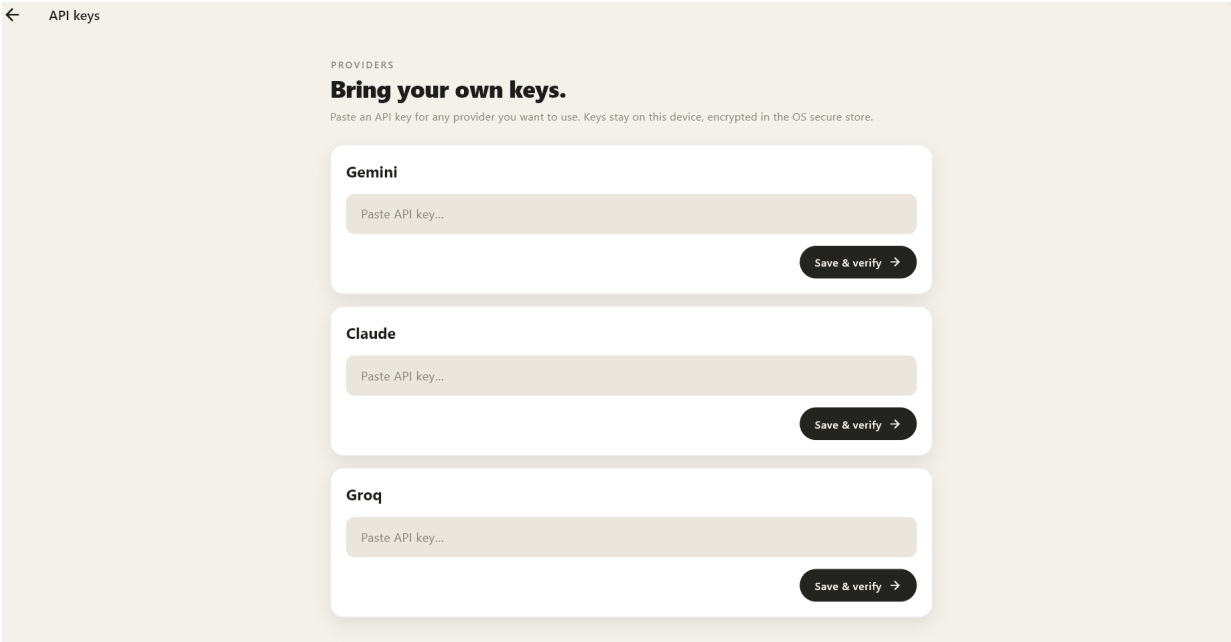
Magisor is under active, daily development, so code-level stats like commit counts and line totals shift constantly and would date this document almost immediately. What's below is the part that doesn't move as fast.

| Metric         | Value                                              |
|----------------|----------------------------------------------------|
| Timeline       | ~7 weeks so far: 2026-05-21 to 2026-07-03, ongoing |
| AI providers   | 3: Gemini, Claude, Groq                            |
| Vision models  | 10+, switchable per provider                       |
| Installer size | ~19 MB                                             |
| Idle RAM       | ~370 MB                                            |



The History tab: populates automatically as you use Ask, Summarize, Circle-to-Search, or Select Text.

The Saved tab: starring any history entry keeps it here, independent of history clears.



The API-key card stack: Gemini, Claude, and Groq, each with a password field and a Save & verify button.

## Building it with Claude Code

This project was built with significant AI assistance, and that claim is more specific than it sounds. “AI wrote the code” and “AI assisted the code” are easy to conflate, and worth separating here.

### Ideation and product decisions: human

The original roadmap, the phased Magic Pointer plan, the v1.6 design language, and all three decisions in Section 03 came from direction and hands-on testing, not from a suggestion an AI made. The same is true of the call to drop the UI Automation phase once it stopped making sense, and the response when GitHub's secret scanner flagged a leaked API key.

### Implementation: Claude Code

Claude Code was the implementation tool across every phase of this build, from the Flutter rewrite through the v1.6 redesign, across many separate sessions. Each session worked against a maintained specification, first `magisor_project_roadmap.md`, later `magic_pointer_spec.md`, as the standing source of truth. The quality of what came out tracked the quality of that spec directly: vague direction produced generic results, while a written, phase-by-phase plan produced a working native OCR bridge on the first real attempt.

### What the collaboration looked like

The loop that shipped this was: write or update the spec, implement one phase, test and steer by actually using the app, commit, move to the next phase. A few concrete examples: when the overlay rendered pink, describing exactly that led to the fix, an unset alpha channel from the native BitBlt capture. When the shake gesture was hair-trigger, saying so got thresholds raised app-wide. When the UI Automation phase turned out not to compose with the freeze model, Claude Code surfaced the conflict directly, and the call to drop it was made by hand.

### The AI excelled at:

- Native/Flutter integration work: a C++/WinRT OCR bridge with a worker thread and hand-marshaled messaging back to the UI thread, correct on the first real attempt.
- Large, coherent refactors across many files at once: the entire v1.6 visual redesign touched the theme and every screen in one pass without regressions.
- Diagnosing bug reports that turned out to be something other than a defect, like an “AI is broken” report that traced back to Gemini's free-tier rate limit.

### The AI did not:

- Choose the freeze-the-screen architecture, or decide to drop the UI Automation phase once it conflicted with that architecture.
- Decide on-device OCR over a cloud vision call, or bring-your-own-key over a hosted backend.
- Run the security response when a leaked key was flagged; that was a same-day audit, directed and verified by hand.



- Make any UX judgment that required actually using the app.

*The implementation velocity came from the tooling. The product decisions did not.*

---

## SECTION 06

# What's next

- Friendlier rate-limit handling, with clear messaging and auto-retry when a free-tier provider throttles a request.
- A macOS port. The entire Dart layer and every service class carry over unchanged; only the native runner (screen capture, OCR, mouse hook) needs a platform-specific rewrite, since it was built behind clean method-channel boundaries from day one.
- iOS and Android companion apps, for cross-device Ask and history sync.
- A live “point” mode: a second, non-freeze interaction where UI Automation reads live content under the pointer. This is the piece dropped from the freeze-model spec, resurfacing as its own interaction.
- Local model support (Ollama), for users who want zero cloud calls at all.
- A rolling context buffer, so “what did that error say?” works even after the dialog is gone.

---

# Links

- **Repository:** [github.com/vinamrapandey/Magisor](https://github.com/vinamrapandey/Magisor)
- **Releases:** [github.com/vinamrapandey/Magisor/releases](https://github.com/vinamrapandey/Magisor/releases)
- **Latest installer:** Magisor-Setup-1.6.0.exe (Windows 10/11 x64)